MACSE501
DSA LAB 02
ASSIGNMENT 02

Name : Yuvaraj R
Reg No : 26MCS1019

Q1.

Problem Statement

Jake, a sales representative, recorded the number of sales made each week of the month. He needs to sort the sales figures at odd positions in descending order and those at even positions in ascending order using insertion sort. This sorting will help him analyze the performance trends effectively.

Example

Input:
10
25 36 96 58 74 14 35 15 75 95
Output:
96 14 75 15 74 36 35 58 25 95

Explanation
Odd positions (indices 0, 2, 4, 6, 8) in descending order: 25, 96, 74, 35, 75 becomes [96, 75, 74, 35, 25].
Even positions (indices 1, 3, 5, 7, 9) in ascending order: 36, 58, 14, 15, 95 becomes [14, 15, 36, 58, 95].
Combine the sorted odd and even positions: [96, 14, 75, 15, 74, 36, 35, 58, 25, 95].

Input Format
The first line contains a single integer **N**, representing the number of weeks.
The second line contains **N** space-separated integers, representing the number of sales made each week.

Output Format
The output displays the sorted array, where the sales numbers at odd positions (1st, 3rd, 5th, etc.) are sorted in descending order and those at even positions (2nd, 4th, 6th, etc.) are sorted in ascending order.

Refer to the sample output for formatting specifications.

Constraints
In this scenario, the test cases fall under the following constraints:
$1 \leq N \leq 20$
$1 \leq$ number of sales ≤ 100

Sample Input Sample Output

```
10
25 36 96 58 74 14 35 15 75 95
96 14 75 15 74 36 35 58 25 95
```

Sample Input Sample Output

```
4
10 20 30 40
30 20 10 40
```

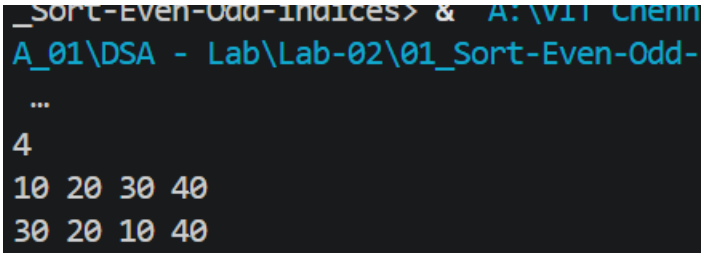
Time Limit: - ms Memory Limit: - kb Code Size: - kb

CODE :

```
#include <stdio.h>

void insertionSortAscending(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

void insertionSortDescending(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] < key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

int main() {

    int n;
```

```
    scanf("%d", &n);

    int *arr = malloc(n * sizeof(int));

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int *odd= malloc(((n + 1) / 2) * sizeof(int));
    int *even =malloc((n / 2) * sizeof(int));

    int oi = 0, ei = 0;

    // Separate odd and even positions
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0)
            odd[oi++] = arr[i];
        else
            even[ei++] = arr[i];
    }

    insertionSortDescending(odd, oi);
    insertionSortAscending(even, ei);

    oi = 0;
    ei = 0;

    // Merge back
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0)
            arr[i] = odd[oi++];
        else
            arr[i] = even[ei++];
    }

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

OUTPUT:



Q2.

Problem statement

Sarah is a historian working on organizing a series of historical events that are recorded only by their occurrence dates. She needs a program that can sort these dates in ascending order to help her understand the chronological progression of these events. Each date is given as three integers representing the day, month, and year.

Your task is to develop a program that reads a list of dates and outputs them sorted from the earliest to the latest using **selection sort**.

Input Format

The first line consists of an integer **N,** representing the number of dates.

The following **N** lines consist of the respective dates as space-separated integers in **dd mm yyyy** format.

Output Format

The output prints **N** lines, each containing three integers day, month, and year, separated by spaces, representing the dates sorted in ascending order.

Refer to the sample output for the formatting specifications.

Constraints

In this scenario, the test cases fall under the following constraints:

$1 \leq N \leq 20$

Sample Input Sample Output

```
5
20 04 2016
15 06 2018
11 09 2019
30 04 1992
16 11 2000
30 4 1992
16 11 2000
20 4 2016
15 6 2018
11 9 2019
```

Sample Input Sample Output

```
3
```

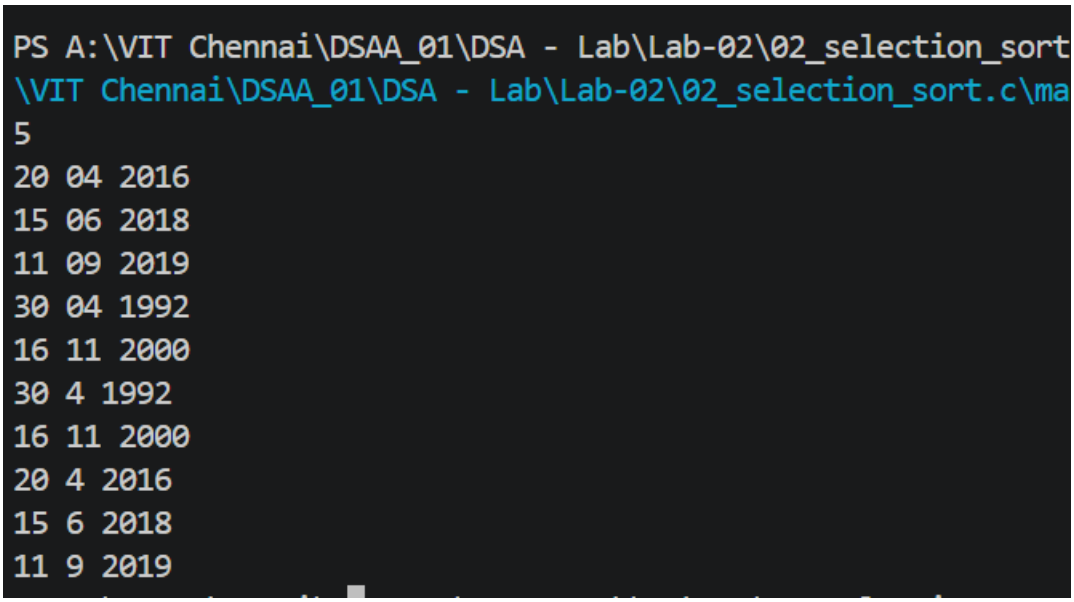
```
26 02 2001
02 12 2000
25 02 2001
2 12 2000
25 2 2001
26 2 2001
```

CODE:

```
#include <stdio.h>

struct Date {
   int day, month, year;
};

int main() {
   int n;
   scanf("%d", &n);

   struct Date d[20], temp;

   for (int i = 0; i < n; i++) {
      scanf("%d %d %d", &d[i].day, &d[i].month, &d[i].year);
   }

   // Selection Sort
   for (int i = 0; i < n - 1; i++) {
      int min = i;

      for (int j = i + 1; j < n; j++) {
         if (d[j].year < d[min].year ||
           (d[j].year == d[min].year && d[j].month < d[min].month) ||
           (d[j].year == d[min].year && d[j].month == d[min].month && d[j].day < d[min].day)) {
            min = j;
         }
      }

      if (min != i) {
         temp = d[i];
         d[i] = d[min];
         d[min] = temp;
      }
   }
```

```
    // Output
    for (int i = 0; i < n; i++) {
        printf("%d %d %d\n", d[i].day, d[i].month, d[i].year);
    }

    return 0;
}
```

OUTPUT:



Q3.

Problem Statement

Hemanth is focused on creating a program to organize a list of employees based on their employee IDs.

Assist him in implementing a function to sort the employee IDs using the bubble sort algorithm. The sorting should prioritize even-numbered employee IDs first, followed by odd-numbered employee IDs.

Display the sorted list of employee IDs according to the specified condition.

Input Format

The first line of input consists of an integer **N,** representing the total number of employees.

The second line consists of **N** space-separated integers, representing the employee IDs.

Output Format

The output prints the even-numbered employee IDs first, followed by odd-numbered employee IDs while maintaining the same order as the input.

Refer to the sample output for formatting specifications.

Constraints

In this scenario, the test cases fall under the following constraints:

1 ≤ N ≤ 20

1 ≤ employee ID ≤ 100

Sample Input Sample Output

```
6
10 11 12 13 14 15
10 12 14 11 13 15
```

Sample Input Sample Output

```
7
71 78 79 87 81 70 74
78 70 74 71 79 87 81
```

Sample Input Sample Output

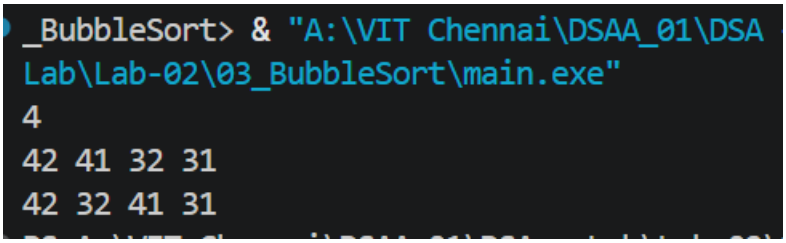
```
4
42 41 32 31
42 32 41 31
```

CODE:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if ((a[j] % 2 != 0) && (a[j + 1] % 2 == 0)) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", a[i]);
    }

    return 0;
}
```

OUTPUT :



Q4.

Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer **n,** representing the number of student records.

For each student record, the following lines consist of name (string), GPA (float), age (integer), and major (string).

The last line of input consists of the sorting criterion ("gpa", "age", or "major").

Output Format

The output displays a sorted list of student records in a tabular format based on the chosen criterion. Round off GPA value to two decimal places.

Note: Use the following to print the header of the table: "Name\t\tGPA\tAge\t\tMajor"

Refer to the sample output for formatting specifications.

Constraints

1 ≤ n ≤ 10

1 ≤ age ≤ 100

1.00 ≤ GPA ≤ 10.00

Sample Input Sample Output

```
4
Harish
```

```
7.8
23
ece
Trell
8
21
cse
Kamalesh
7
25
it
Maaran
5.6
20
civil
age
Sorted Student Records:
Name           GPA    Age            Major
Maaran         5.60   20             civil
Trell          8.00   21             cse
Harish         7.80   23             ece
Kamalesh       7.00   25             it
```

Sample Input Sample Output

```
4
Harish
7.8
23
ece
Trell
8
21
cse
Kamalesh
7
25
it
Maaran
5.6
20
civil
gpa
Sorted Student Records:
Name           GPA    Age            Major
```

```
Maaran          5.60    20              civil
Kamalesh        7.00    25              it
Harish          7.80    23              ece
Trell           8.00    21              cse
```

Sample Input Sample Output

```
4
Harish
7.8
23
ece
Trell
8
21
cse
Kamalesh
7
25
it
Maaran
5.6
20
civil
major
Sorted Student Records:
Name            GPA     Age             Major
Maaran          5.60    20              civil
Trell           8.00    21              cse
Harish          7.80    23              ece
Kamalesh        7.00    25              it
```

CODE:

```
#include <stdio.h>
#include <string.h>

struct Student {
   char name[20], major[20];
   float gpa;
   int age;
};
```

```
void swap(struct Student *a, struct Student *b) {
    struct Student t = *a;
    *a = *b;
    *b = t;
}

int partition(struct Student s[], int low, int high, char key[]) {
    int i = low - 1;

    for (int j = low; j < high; j++) {
        int cond = 0;

        if (strcmp(key, "gpa") == 0)
            cond = s[j].gpa < s[high].gpa;
        else if (strcmp(key, "age") == 0)
            cond = s[j].age < s[high].age;
        else
            cond = strcmp(s[j].major, s[high].major) < 0;

        if (cond) {
            i++;
            swap(&s[i], &s[j]);
        }
    }

    swap(&s[i + 1], &s[high]);
    return i + 1;
}

void quicksort(struct Student s[], int low, int high, char key[]) {
    if (low < high) {
        int p = partition(s, low, high, key);
        quicksort(s, low, p - 1, key);
        quicksort(s, p + 1, high, key);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    struct Student s[10];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%s", s[i].name);
        scanf("%f", &s[i].gpa);
        scanf("%d", &s[i].age);
        scanf("%s", s[i].major);
    }

    char key[10];
    scanf("%s", key);

    quicksort(s, 0, n - 1, key);

    printf("Sorted Student Records:\n");
    printf("Name\t\tGPA\tAge\t\tMajor\n");

    for (int i = 0; i < n; i++) {
        printf("%-10s\t%.2f\t%d\t\t%s\n",
               s[i].name, s[i].gpa, s[i].age, s[i].major);
    }

    return 0;
}
```

OUTPUT:

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-02> & "A:\VIT C
\DSA - Lab\Lab-02\04_QuickSort.exe"
4
harish
7.8
23
ece
Trell
8
21
cse
kamlesh
7
25
it
maaran
5.6
20
civil
age
Sorted Student Records:
Name            GPA     Age             Major
maaran          5.60    20              civil
Trell           8.00    21              cse
harish          7.80    23              ece
kamlesh         7.00    25              it
```

Q5.

Problem Statement

A computer science society of a college asks students to register for a workshop organized by them. The final list of registered students consists of College ID.

Anand, the organizer needs to sort that list using Quick Sort and print the PIVOT element after every sorting pass. Also, some of the students have registered more than once. So he needs to remove the duplicate IDs as well (Remove the duplicate after sorting).

Help Anand to complete this task.

Input Format

The first line contains an integer **n**, the number of students.

The next **n** lines contain integers, each representing the IDs of the students.

Output Format

For each pass of the Quick Sort algorithm, output the pivot used in that pass.
After sorting and removing duplicates, output the final array.

Refer to the sample output for formatting specifications.

Constraints

2 ≤ n ≤ 50

Sample Input Sample Output

```
6
5
4
3
1
2
1
Pivot for pass 1: 1
Pivot for pass 2: 4
Pivot for pass 3: 2
Final array: 1 2 3 4 5
```

Sample Input Sample Output

```
3
1
1
1
Pivot for pass 1: 1
Pivot for pass 2: 1
Final array: 1
```

CODE:

```
#include <stdio.h>

void swap(int *a, int *b) {
   int t = *a;
   *a = *b;
   *b = t;
}

int partition(int a[], int low, int high, int *pass) {
   int pivot = a[high];
   int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }

    swap(&a[i + 1], &a[high]);

    printf("Pivot for pass %d: %d\n", (*pass)++, pivot);

    return i + 1;
}

void quicksort(int a[], int low, int high, int *pass) {
    if (low < high) {
        int p = partition(a, low, high, pass);
        quicksort(a, low, p - 1, pass);
        quicksort(a, p + 1, high, pass);
    }
}

int main() {
    int n, pass = 1;
    scanf("%d", &n);

    int a[50];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quicksort(a, 0, n - 1, &pass);

    printf("Final array: ");
    printf("%d ", a[0]);

    for (int i = 1; i < n; i++) {
        if (a[i] != a[i - 1])
            printf("%d ", a[i]);
    }

    return 0;
}
```

OUTPUT:

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-02>
 "A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-02\0
quick_sort_duplicate_removal.exe"
3
1
1
1
Pivot for pass 1: 1
Pivot for pass 2: 1
Final array: 1
```

Q6.

Problem Statement

You are working as a data analyst at a research institute. As part of a data analysis project, you are given an array of integers representing various data points collected during an experiment. Your task is to sort the array in increasing order based on the frequency of the values.

Here's how the sorting should be done:

1. First, count the frequency of each unique integer in the array.
2. Sort the integers based on their frequencies in increasing order. If two or more integers have the same frequency, sort them in decreasing order.

Write a program using the **Merge Sort** algorithm to sort the array based on the specified conditions and return the sorted array.

Example 1

Input:
array = [1,1,2,2,2,3]

Output:
3 1 1 2 2 2

Explanation:
'3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.

Example 2

Input:
array = [2,3,1,3,2]

Output:
1 3 3 2 2

Explanation:
'2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
Input Format
The first line of input consists of the number of integers **n.**
The second line of input consists of **n** integers, separated by space.
Output Format
The output prints the sorted array in increasing order based on the frequency of the values.

Refer to the sample output for formatting specifications.
Constraints
In this scenario, the test cases fall under the following constraints:
2 ≤ n ≤ 30
1 ≤ elements of the array ≤ 100
Sample Input Sample Output

```
6
1 1 2 2 2 3
3 1 1 2 2 2
```

Sample Input Sample Output

```
5
2 3 1 3 2
1 3 3 2 2
```

Sample Input Sample Output

```
5
4 5 6 4 0
6 5 0 4 4
```

Time Limit: - ms Memory Limit: - kb Code Size: - kb

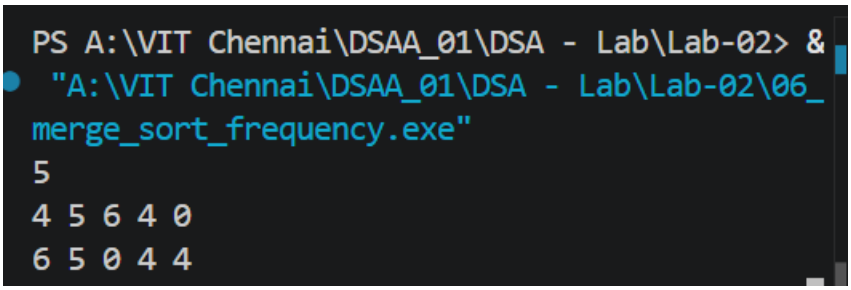
CODE:

```
#include <stdio.h>

int freq[101];

void merge(int a[], int l, int m, int r) {
   int temp[30];
   int i = l, j = m + 1, k = 0;

   while (i <= m && j <= r) {
      if (freq[a[i]] < freq[a[j]])
         temp[k++] = a[i++];
```

```
        else if (freq[a[i]] > freq[a[j]])
            temp[k++] = a[j++];
        else {
            if (a[i] > a[j])      // Same frequency -> larger value first
                temp[k++] = a[i++];
            else
                temp[k++] = a[j++];
        }
    }

    while (i <= m)
        temp[k++] = a[i++];

    while (j <= r)
        temp[k++] = a[j++];

    for (i = l, k = 0; i <= r; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int mid = (l + r) / 2;
        mergeSort(a, l, mid);
        mergeSort(a, mid + 1, r);
        merge(a, l, mid, r);
    }
}

int main() {
    int n, a[30];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        freq[a[i]]++;
    }

    mergeSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
```

```
    return 0;
}
```

OUTPUT:



Q7.

Problem Statement

Imagine you are working on a system that handles a large number of student records for different subjects in a university. Each subject's student records are stored as sorted arrays based on the student's ID. The system needs to merge the sorted arrays into a single sorted array to perform further analysis and generate reports.

You are asked to implement a function that takes the number of subjects **k** and their corresponding sorted arrays as input and returns a single sorted array containing all the student records merged together.

Input Format

The first line contains an integer **k**, the number of subjects.
The next **k** lines contain **k** integers each, representing student IDs in a sorted array.

Output Format

The output prints a single line containing the merged and sorted student IDs.

Refer to the sample output for formatting specifications.

Constraints

In this scenario, the test cases fall under the following constraints:
1 ≤ k ≤ 20
1 ≤ student ID ≤ 100

Sample Input Sample Output

```
3
1 4 7
2 5 8
3 6 9
Merged and Sorted Array: 1 2 3 4 5 6 7 8 9
```

Sample Input Sample Output

```
2
1 7
3 4
```

```
Merged and Sorted Array: 1 3 4 7
```

Sample Input Sample Output

```
3
1 2 3
2 3 4
3 4 5
Merged and Sorted Array: 1 2 2 3 3 3 4 4 5
```

CODE:

```
#include <stdio.h>

int main() {
    int k;
    scanf("%d", &k);

    int a[20][20];
    int index[20] = {0};

    // Read the k x k matrix
    for (int i = 0; i < k; i++)
        for (int j = 0; j < k; j++)
            scanf("%d", &a[i][j]);

    printf("Merged and Sorted Array: ");

    // Merge k sorted arrays
    for (int count = 0; count < k * k; count++) {
        int min = 101, row = -1;

        for (int i = 0; i < k; i++) {
            if (index[i] < k && a[i][index[i]] < min) {
                min = a[i][index[i]];
                row = i;
            }
        }

        printf("%d ", min);
        index[row]++;
    }

    return 0;
}
```

OUTPUT

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-02> &
 "A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-02\07_
merge_k_sorted_arrays.exe"
3
1 4 7
2 5 8
3 6 9
Merged and Sorted Array: 1 2 3 4 5 6 7 8 9
```

Q8.

Problem Statement

Joel is a data analyst working for a manufacturing company. His task is to update the layout of the factory floor, which is represented by a matrix of integers. Each cell in the matrix represents a section of the factory floor, with some sections containing machinery (represented by zeros) and others being empty.

Joel needs to modify the factory floor matrix based on the following rules:

1. If any cell in a row contains a zero, all cells in that row should be set to zero.
2. If any cell in a column contains a zero, all cells in that column should be set to zero.

Input Format

The first line consists of two integers **r** and **c**, separated by a space, representing the number of rows and columns in the matrix.

The next **r** lines contain **c** space-separated integers, representing the elements of the matrix.

Output Format

The output prints the modified matrix where all rows and columns containing at least one zero are entirely set to zero.

Refer to the sample output for formatting specifications.

Constraints

In the given scenario, the test cases fall under the following constraints:

$2 \le r, c \le 8$

$0 \le \text{elements} \le 9$

Sample Input Sample Output

```
2 2
1 0
4 9
0 0
4 0
```

Sample Input Sample Output

```
3 4
1 0 2 3
0 4 5 6
7 8 6 9
0 0 0 0
0 0 0 0
0 0 6 9
```

Sample Input Sample Output

```
8 8
1 0 2 3 7 8 6 2
0 4 5 6 4 2 3 2
7 8 0 9 2 3 4 7
2 3 8 0 7 8 4 6
1 2 3 4 5 6 7 8
2 5 8 7 4 1 3 4
2 4 6 9 7 8 1 3
0 1 4 7 2 3 7 3
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 5 6 7 8
0 0 0 0 4 1 3 4
0 0 0 0 7 8 1 3
0 0 0 0 0 0 0 0
```

Sample Input Sample Output

```
2 3
2 5 7
1 2 4
2 5 7
1 2 4
```

CODE:

```
#include <stdio.h>

int main() {
    int r, c;
    scanf("%d %d", &r, &c);

    int a[8][8];
    int row[8] = {0}, col[8] = {0};
```

```
    // Read matrix
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j] == 0) {
                row[i] = 1;
                col[j] = 1;
            }
        }
    }

    // Set rows and columns to zero
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            if (row[i] || col[j])
                a[i][j] = 0;
        }
    }

    // Print matrix
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT :

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-0
 "A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-02
set_matrix_zeroes.exe"
2 2
1 0
4 9
0 0
4 0
```

Q9.

Problem Statement

In a vibrant town, the villagers have a unique tradition to celebrate their birthdays. Instead of celebrating with age, they celebrate with the concept of a "digital root." The digital root of a number is obtained by repeatedly summing its digits until a single-digit number is obtained.

You are tasked with writing a program to help the villagers determine their digital root. The program should accept a positive integer representing a villager's age and output their digital root.

Digital Root: The single-digit value obtained by recursively summing the digits of the number until a single-digit result is obtained. For example, **i**f the given number is 1234, the sum of 1+2+3+4 = 10. So the digital root will be 1+0 = 1.

Input Format

The input consists of an integer **N,** representing the age of a villager.
Here age ranges from 1 to 10^5.

Output Format

The output prints an integer, which is the sum of digits for **N** until a single digit is obtained.

Refer to the sample output for the formatting specifications.

Constraints

In this scenario, the test cases fall under the following constraints:
$1 \le N \le 10^5$

Sample Input Sample Output

```
5678
8
```

Sample Input Sample Output

```
1234
1
```

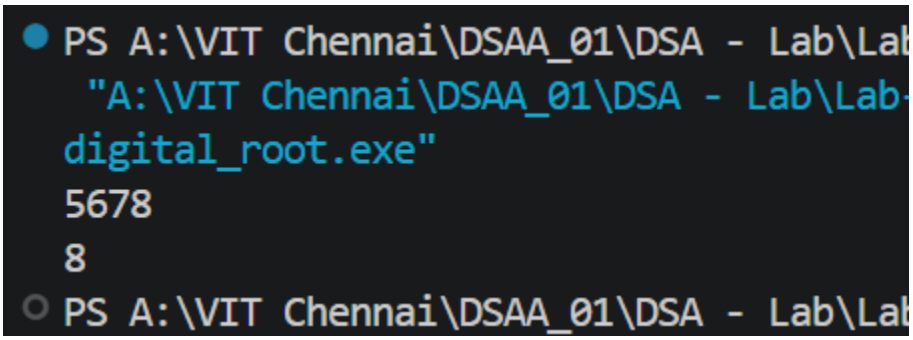
Time Limit: - ms Memory Limit: - kb Code Size: - kb

CODE:

```
#include <stdio.h>

int main() {
   int n, sum;

   scanf("%d", &n);

   while (n >= 10) {
      sum = 0;
```

```
        while (n > 0) {
            sum += n % 10;
            n /= 10;
        }
        n = sum;
    }

    printf("%d", n);

    return 0;
}
```

OUTPUT:



Q10.

Problem Statement

In a bustling city, there is an annual festival known as the "Festival of Prizes," where attendees can win exciting rewards based on their ticket numbers. This year, the festival organizers introduced a unique game to determine if a ticket number is "lucky."

A ticket number is deemed lucky if the sum of the digits of its square equals the original ticket number. Participants are eager to find out if their ticket numbers are lucky, as only lucky ticket holders will be eligible for special prizes.

You are tasked with writing a program that checks if a given ticket number is lucky. Use a recursive function.

Input Format

The input consists of a single positive integer **N**.

Output Format

If the ticket number is lucky, print: "Yes, N is a lucky number".
If the ticket number is not lucky, print: "No, N is not a lucky number".

Refer to the sample output for formatting specifications.

Constraints

In this scenario, the test cases fall under the following constraints:
$1 \leq N \leq 100$

Sample Input Sample Output

```
9
Yes, 9 is a lucky number
```

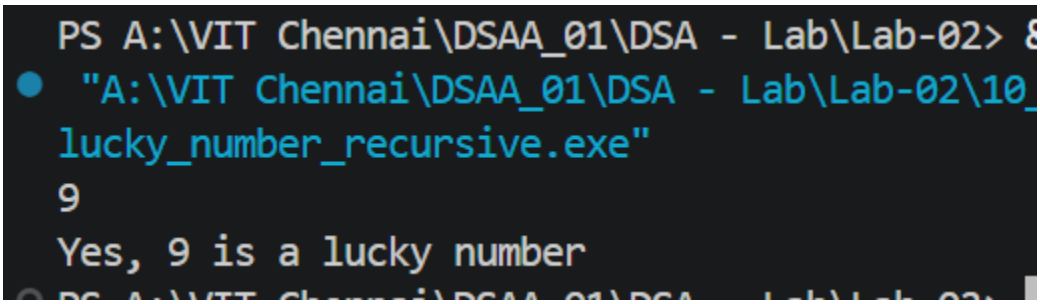
Sample Input Sample Output

```
25
No, 25 is not a lucky number
```

CODE:

```
#include <stdio.h>

int sumDigits(int n) {
   if (n == 0)
      return 0;
   return (n % 10) + sumDigits(n / 10);
}

int main() {
   int n;
   scanf("%d", &n);

   int square = n * n;

   if (sumDigits(square) == n)
      printf("Yes, %d is a lucky number", n);
   else
      printf("No, %d is not a lucky number", n);

   return 0;
}
```

OUTPUT:



Q11.

Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

Company Tags: Microsoft, Adobe

Input Format

The first line contains an integer **n**, the number of different coin denominations.
The second line contains **n** space-separated integers representing the coin denominations.
The third line contains an integer **m**, the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Constraints

In this scenario, the test cases fall under the following constraints:
$2 \leq n \leq 100$
$1 \leq m \leq 100$

Sample Input Sample Output

```
3
1 2 3
4
4
```

Sample Input Sample Output

```
4
2 5 3 6
10
5
```

CODE:

```
#include <stdio.h>

int main() {

   int n, m;

   scanf("%d", &n);
```

```
    int coin[100];
    for (int i = 0; i < n; i++)
        scanf("%d", &coin[i]);
    scanf("%d", &m);
    int dp[101] = {0};
    dp[0] = 1;

    for (int i = 0; i < n; i++) {
        for (int j = coin[i]; j <= m; j++) {
            dp[j] += dp[j - coin[i]];
        }
    }
    printf("%d", dp[m]);
    return 0;
}
```

OUTPUT: